

Flatcar Reference and Developer Guides

Supply Chain Security (SLSA)

Flatcar self-assesses at **SLSA Level 3**, working toward Level 4. Key properties:

Requirement	Status
Version-controlled source	✓
Verified source history	✓
Two-person review	✓
Scripted, service-based build	✓
Build-as-code, ephemeral, isolated	✓
Provenance available, authenticated, non-falsifiable	✓
Dependencies complete	✓
Hermetic builds	Partial (infrastructure doesn't enforce network isolation yet)
Security controls / superuser restrictions	Partial

Runtime verification: Flatcar images are GPG-signed. The OS partition uses dm-verity to detect tampering at boot. An SPDX SBOM is published with each release.

SDK and Developer Guides

The Flatcar SDK is a container-based build environment for building and modifying Flatcar OS images.

Guide	Topic
SDK bootstrapping	Setting up the build environment
SDK — modifying Flatcar	Patching packages, overlays
SDK — building production images	Full image build process
SDK — disk partitions	Partition layout details
SDK — tips and tricks	Common development patterns
Kernel modules	Building and loading out-of-tree modules

Guide	Topic
Release guide	Cutting a new Flatcar release

Disk partition layout:

Partition	Purpose
EFI System Partition	GRUB bootloader, <code>flatcar/first_boot</code> flag
USR-A	Active OS image (<code>/usr</code>)
USR-B	Standby OS image (updates applied here)
ROOT-A	Active root filesystem
ROOT-B	Standby root filesystem
OEM	OEM-specific tools (now sysexec images since 3760.0.0)

Integrations

The `reference/integrations` doc lists supported integrations including:

- Ansible roles for Flatcar
- Terraform providers
- Packer builders
- Various cloud-init and Ignition tooling

Constants and IDs

`reference/constants-and-ids` documents the well-known GUIDs and IDs used in Flatcar:

- Disk partition type GUIDs
- OEM IDs
- Platform identifiers used by Ignition (`flatcar.oem.id`)

Self-Paced Learning Series

Four sessions covering Flatcar from basics to advanced:

Session	Topic	Prerequisites
Basics and Testing	Local test environment, first Butane config, QEMU VM	None
Advanced Provisioning	Complex deployments, node config management	Session 1

Session	Topic	Prerequisites
Managing Storage	Additional storage, LUKS encryption	Session 1
Immutability, Updates, Rollbacks	Boot process deep dive, in-place upgrades, automated rollbacks, channels	Sessions 1–2

Migrating from CoreOS Container Linux

Flatcar is a drop-in replacement. Migration paths:

In-place update (existing CoreOS CL cluster):

```
# Run on each CoreOS node
sudo /usr/share/oem/flatcar-update.sh
```

Fresh install: Deploy Flatcar directly. Config format is compatible with minor naming changes (see [flatcar-installing](#)).

Key differences to address:

Area	CoreOS	Flatcar
Installer	coreos-installer	flatcar-installer
First-boot flag	coreos.first_boot=1	flatcar.first_boot=1
Config URL	coreos.config.url	ignition.config.url
OEM ID	coreos.oem.id	flatcar.oem.id
QEMU fw_cfg key	opt/com.coreos/config	opt/org.flatcar-linux/config
VMware guestinfo	coreos.config.data	ignition.config.data
GRUB config file	/boot/coreos/first_boot	/boot/flatcar/first_boot

Both the old `coreos.*` and new `flatcar.*` kernel parameters are supported in current releases for backward compatibility during migration.